

PEB 遍历的蓝队检测视角

前几课我们一直在扮演红队：隐藏 IAT、封装哈希解析、用 IDA 验证隐蔽性。但任何攻击技术，如果不了解它如何被检测，就等于在盲打。

本课我们切换视角，站在蓝队（防御方）立场，分析 PEB 遍历、gs:[0x60] 访问、调试器附着这些行为的可检测性，并给出对应的反制策略。

● 蓝队视角：PEB 遍历为什么可疑

正常程序几乎从不手工遍历 PEB。它们用 GetModuleHandle、GetProcAddress 这些官方 API。只有当程序试图绕过 API 监控时，才会去直接访问 gs:[0x60]。

蓝队关注的核心问题：

- 谁在读 gs:[0x60]？
- 读取之后，它拿 PEB 做了什么？
- 它的调用栈长什么样？

● 检测手段一：异常访问模式

(一) 基于 ETW 的内存访问监控

Windows 的 ETW（Event Tracing for Windows）可以追踪内存访问。如果 EDR 开启了内存监控，它可以记录：

- 访问 gs:[0x60] 的进程 ID、线程 ID。
- 访问发生的代码地址（RIP）。
- 访问的频率 和 上下文。

可疑特征：

- 一个普通的用户态进程，在短时间内多次访问 gs:[0x60]。
- 访问后紧接着执行导出表遍历（AddressOfNames 循环）。
- 访问的代码地址位于非系统模块（比如你自己的 .exe 里）。

(二) 基于调用栈的分析

EDR 可以通过栈回溯（Stack Walking）判断一个函数的调用者是谁：

- 如果 GetModuleAddress 是从 main 直接调用的，那它的调用栈很短，看起来很可疑。
- 如果它是从某个“正常流程”中间接调用的，看起来就自然得多。

蓝队逻辑：一个普通程序，不应该在启动 100 毫秒内就遍历 PEB 链表。这更像是一个 loader 或注入器在“自举”。

(三) 基于内存扫描的检测

EDR 可以扫描进程内存，寻找已知的 PEB 遍历代码特征：

- 特征 1：mov rax, gs:[0x60] 这条指令。
- 特征 2：CONTAINING_RECORD 附近的指针减法模式（比如 pEntry - 0x10）。
- 特征 3：对 BaseDllName 的字符串哈希计算循环。

● 检测手段二：调试器附着检测

这是蓝队最常用的“反调试”手段之一。

(一) NtQueryInformationProcess + ProcessDebugPort

```
DWORD debugPort = 0;
NtQueryInformationProcess(
    GetCurrentProcess(),
    ProcessDebugPort,      // 参数值：7
    &debugPort,
    sizeof(debugPort),
    NULL
);
```

```
if (debugPort != 0) {  
    // 当前进程被调试器附着  
}
```

物理意义：ProcessDebugPort 返回非 0，说明当前进程的“调试端口”被占用了，意味着有调试器（比如 x64dbg、WinDbg）正在挂在你身上。

(二) 蓝队如何利用它

蓝队在做反调试时，会在关键逻辑前插入这段检测。如果发现被调试，就：

- 改变行为（比如不执行注入）。
- 触发异常，让调试器崩溃。
- 记录日志，上报可疑行为。

(三) 红队如何反制

- 检测自身是否被调试：先自己查 ProcessDebugPort，如果非 0，说明被跟踪了，可以提前退出或改变行为。
- Hook 掉 NtQueryInformationProcess：把返回值改成 0，欺骗蓝队的检测逻辑。
- 通过直接系统调用绕过：不用 ntdll.dll 的 NtQueryInformationProcess，自己发 syscall 指令。

● 反制策略一：拆分遍历逻辑

问题：GetModuleAddress 函数把“读 PEB → 遍历链表 → 算哈希 → 返回基址”四步全做完了。这个函数本身就是一个完整的攻击特征。

反制思路：把这四步拆成多个小函数，分散到不同位置。

```
// 第一步：只负责读 PEB 和 Ldr  
PEB_LDR_DATA* GetLdr() {  
    PEB* pPeb = (PEB*)__readgsqword(0x60);  
    return (PEB_LDR_DATA*)pPeb->Ldr;  
}  
  
// 第二步：只负责遍历链表，返回节点  
LIST_ENTRY* FindModuleEntry(PEB_LDR_DATA* pLdr, DWORD hash) {  
    // ...  
}  
  
// 第三步：只负责提取基址  
PVOID ExtractBase(LIST_ENTRY* pEntry) {  
    // ...  
}  
  
// 第四步：只负责算哈希比对  
bool CheckHash(UNICODE_STRING* name, DWORD hash) {  
    // ...  
}  
  
// 组装：把四步串起来  
void* GetModuleAddress(DWORD hash) {  
    PEB_LDR_DATA* pLdr = GetLdr();  
    LIST_ENTRY* pEntry = FindModuleEntry(pLdr, hash);  
    return ExtractBase(pEntry);  
}
```

效果：

- 单函数内不再有明显完整的“PEB 遍历”特征。
- 调用栈变复杂，蓝队更难判断整体意图。
- 编译器优化后，小函数可能被内联，所以最好加上 `\_\_declspec(noinline)`。

● 反制策略二：使用间接调用

问题：代码是直接 call GetModuleAddress。这在栈回溯里非常明显。

反制思路：用函数指针调用。

```
typedef void* (*pGetModuleAddress)(DWORD);
// 运行时获取函数地址（可以用哈希解析）
pGetModuleAddress func = (pGetModuleAddress)0x...;
// 间接调用
void* result = func(hash);
```

效果：

- 调用栈里显示的是“通过函数指针调用”，而不是“直接调用 GetModuleAddress”。
- 蓝队的静态特征扫描会失效（因为调用点没有函数名）。

● 反制策略三：通过异常处理隐藏 PEB 访问

问题：gs:[0x60]这条指令可能被 EDR 通过硬件断点或内存扫描捕获。

反制思路：把 PEB 访问放在__try / __except 里，用 SEH（结构化异常处理）包裹。

```
void* StealthGetPeb() {
    void* pPeb = nullptr;
    __try {
        pPeb = (void*)__readgsqword(0x60);
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        // 如果访问被拦截，走这里
        return nullptr;
    }
    return pPeb;
}
```

效果：

- 如果 EDR 在 gs:[0x60] 上设置了硬件断点，异常会被你的 __except 捕获，而不是让进程崩溃。
- 蓝队可能只看到“一次异常”，而不是“一次成功的 PEB 访问”。

注意：这个技巧在现代 EDR 上效果有限（EDR 仍然能记录异常），但它能阻止一些低级的调试器中断。

● 蓝队检测的最终逻辑 总结

检测点	蓝队看什么	红队反制
PEB 访问频率	短时间多次读 gs:[0x60]	拆分逻辑，减少单次调用次数
调用栈深度	从 main 直接调用 GetModuleAddress	使用间接调用、插入中间函数
导出表遍历特征	AddressOfNames 循环 + 哈希计算	用系统 API 替代（但会暴露 IAT）
调试器附着	ProcessDebugPort != 0	检测自身 + Hook NtQueryInformationProcess

内存代码特征	mov rax, gs:[0x60]	用异常处理包裹、使用间接寻址
--------	--------------------	----------------